

FINAL (2022-05-13)

See <https://bit.ly/3KxuMdV> for the official information on the time and location of this exam.

Question 1:

- (A) Order functions in *increasing* order by their asymptotic growth rate: Write them in some sequence $g_1, g_2, \dots, g_8$ such that $g_1 = O(g_2)$, $g_2 = O(g_3)$, $\dots$, $g_7 = O(g_8)$.

$$\begin{aligned} f_1(n) &= n^e, & f_2(n) &= e^n, & f_3(n) &= \binom{n}{n-3}, & f_4(n) &= \sqrt{2\sqrt{n}}, \\ f_5(n) &= \binom{n}{4}, & f_6(n) &= 2^{\log_2^4 n}, & f_7(n) &= n^{3(\log_2 n)^2}, & f_8(n) &= (n-1)(n+1)(n^2+1)(n^4+1). \end{aligned}$$

- (B) Find function $g(n)$ such that $T(n)$ is in $\Theta(g(n))$. Here $T(n)$ is defined by a recurrence:

$$T(n) = \begin{cases} T(\lfloor \frac{n}{3} \rfloor) + T(\lfloor \frac{2n}{3} \rfloor) + n, & \text{if } n > 0, \\ 1, & \text{if } n = 0. \end{cases}$$

Answer:

- (A) To classify the functions, we identify functions with polynomial growth rate (f_1, f_3, f_4, f_5) and order them by the exponent of the polynomial, which is either visible in the expression of that function or can be found by some algebra.

Next, we identify functions growing faster than polynomials (typically, because their exponent is not constant, but growing). They are f_7, f_6, f_4, f_2 . Some of them might seem difficult to compare. Taking logarithm from both sides may help. Note that all polynomial expressions $P(n)$ that are in $\Theta(n^k)$ (regardless of the exponent k) have logarithms $\log_2 P(n)$ that are in $\Theta(k \log_2 n) = \Theta(\log_2 n)$. So, if the logarithm of some function grows faster than $\log_2 n$, then it grows faster than a logarithm.

- $f_1(n) = n^e \approx n^{2.71828}$ (we claim that this is the slowest growing function from the list; it is in $O(n^3)$, observe that for large n it grows slower than the polynomials of the 3rd degree),
- $f_3(n) = \binom{n}{n-3} = \frac{n(n-1)(n-2)}{6} = \Theta(n^3)$ (grows as fast as n^3).
- $f_5(n) = \binom{n}{4} = \frac{n(n-1)(n-2)(n-3)}{24} = \Theta(n^4)$ (grows as fast as n^4).
- $f_8(n) = (n-1)(n+1)(n^2+1)(n^4+1) = \Theta(n^8)$ (grows as a polynomial of 8th degree).
- $f_7(n) = n^{3(\log_2 n)^2}$, since $\log_2 f_7(n) = 3 \log_2^2 n \cdot \log_2 n = \Theta(\log_2^3 n)$ – any polynomial (including $f_8(n)$) will be in $O(f_7(n))$, since the logarithms of polynomials are in $\Theta(\log_2 n)$, so they do not grow as fast as $\Theta(\log_2^3 n)$.
- $f_6(n) = 2^{\log_2^4 n}$, observe that $\log_2 f_6(n) = \log_2 2^{\log_2^4 n} = \log_2^4 n$.
- $f_4(n) = \sqrt{2\sqrt{n}}$, observe that $\log_2 f_4(n) = \log_2 2^{\frac{1}{2}\sqrt{n}} = \frac{1}{2}\sqrt{n}$. Observe also that the square root grows faster than any power of a logarithm. To see that $f_6(n)$ is in $O(f_4(n))$ verify that the ratio of their logarithms $\lim_{n \rightarrow \infty} \frac{\log_2^4 n}{\sqrt{n}} = 0$. (You could try to use L'Hopital's rule to find this limit).

8. $f_2(n) = e^n$. This is the fastest growing function of all, since its logarithm is $\log_2 f_2(n) = \log_2 e^n = n \cdot \log_2 e = \Theta(n)$. This function n grows faster than the square root function from the previous function.

Many more examples how to arrange functions by their asymptotic growth rate (Big-O) can be found in other sources, for example, <https://bit.ly/3HR8yUi>.

- (B) Answer: $T(n) \in \Theta(n \log n)$. To understand the growth rate, compute a few first values of this function:

n	0	1	2	3	4	5	6	7	8	9	10
$T(n)$	1	3	6	12	13	20	25	26	34	46	47

For example, $T(10)$ is computed like this:

$$\begin{aligned} T(10) &= T(3) + T(6) + 10 = (T(1) + T(2) + 3) + (T(2) + T(4) + 6) + 10 = \\ &= T(1) + 2T(2) + T(4) + 19 = (T(0) + T(0) + 1) + 2 \cdot (T(0) + T(1) + 2) + (T(1) + T(2) + 4) + 19 = \\ &= (1 + 1 + 1) + 2 \cdot (1 + 3 + 2) + (3 + 6 + 4) + 19 = 47. \end{aligned}$$

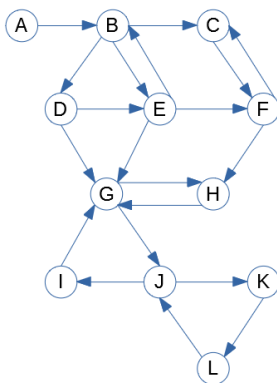
The recurrence is similar to those solvable by Master theorem, <https://bit.ly/3OpaxSI>; one could get such a recurrence for a variant of Mergesort algorithm (which splits an array of n elements into two subarrays of unequal lengths $n/3$ and $2n/3$ respectively). Let us show that the function $T(n)$ is in $O(n \log_2 n)$.

We can apply the recurrence formula certain number of times depending on n . Every time it is applied to all the terms $T(n_i)$, they split into smaller terms (for example, $T(n_i) = T(n'_i) + T(n''_i) + n_i$, where $n'_i + n''_i \approx n_i$ (it may be slightly smaller because we are taking the floor function)).

The total number of times we can multiply some number n with $(2/3)$ until the result is less than 1 is equal to $\log_{3/2} n = \Theta(\log_2 n)$. So the splitting can happen no more than $C \cdot \log_2 n$ times, where C is some positive constant. Every time we split, we also add a number which equals to n (or slightly smaller).

Finally, we get that $T(n)$ is in $O(n \log_2 n)$ since we add quantity $O(n)$ every time we split and we split $O(\log_2 n)$ times. We observe that the regular Merge sort also has the same time complexity.

Question 2: Run Kosaraju algorithm to find strongly connected components.



- (A) Run the DFS on the given graph. (Whenever you run out of traversable nodes and need to pick a new vertex, select the alphabetically first one. Also, when you visit the children of a given node – visit them in an alphabetical order.)

Label each vertex with two numbers d/f, where d is the discovery time, but f is the finishing time during the DFS traversal. (Both d and f are integers from the interval $[1; 2 \cdot 12] = [1; 24]$.)

- (B) As requested by the Kosaraju algorithm, transpose the directed graphs matrix (i.e. flip all the arrows), and run the DFS again. (This time the ordering of the nodes is different: In Kosaraju algorithm they are determined by the d/f numbers found in the previous step. Use this ordering every time when you run out of traversable nodes or need to visit the children nodes.)

In this new graph mark all the strongly connected components you have found. (Components are marked by drawing an oval shape around all the nodes in that component.)

- (C) Estimate the time complexity of this algorithm – find the smallest (slowest growing) $g(n)$ such that the runtime of the algorithm $T(n)$ is in $O(g(n))$.

Question 3: Consider the following sequence of hexadecimal digits:

$$T = 255044462D312E35 \dots$$

The hash function is determined by the following formula:

$$h(a_0a_1a_2 \dots a_{n-1}) = \left(\sum_{k=0}^{n-1} a_k 16^{n-1-k} \right) \bmod 109$$

- (A) Select the window size $s = 6$ and find the hash values of the first two substrings ($P_0 = 255044$ and $P_2 = 550444$) in the given text.

- (B) Describe the algorithm (using constant time) to skip a digit from the start of the pattern. Your algorithm can use the previous hash value and the window size s as inputs.

Run this algorithm to get $h(P_1) = h(55044)$ from $h(P_0) = h(255044)$ skipping the hex digit “2” at the very beginning (assuming that the window size is $s = 6$).

- (C) Describe the algorithm (using constant time) to append a digit to the end of the pattern. Your algorithm can use the previous hash value and the window size s as inputs.

Run your algorithm to get $h(P_2) = h(550444)$ from $h(P_1) = h(55044)$. (Your hash value for $h(P_2)$ should coincide with the value obtained in (A).)

Question 4:

- (A) Build the KMP (Knuth-Morris-Pratt) prefix function to search for this pattern: $P = \text{ababacab}$.

- (B) Show how KMP works to find *all occurrences* of the pattern in the following text: $T = \text{abaababacababa}$.

Namely, write all the letters of this text in a horizontal line. Under this text show multiple copies of the pattern (copied with different offsets so that letters in the pattern are compared to the letters in the text T located directly above it). For each copy of the pattern circle those letters that were compared with the above text.

- (C) How many letter-to-letter comparisons would be made, if P is searched in T using the naive search algorithm? How many were used by the KMP algorithm in (B)?

Answer:

(A) ababacab

(B) DEF

(C) GHI

Question 5: Consider this list of strings:

```
S = ["Croatia", "Iceland", "Ireland", "Denmark", "Bulgaria", "Andorra"]
```

Insert this list into a hash table using the following hash function in Python:

```
hash('ABC') % 8
```

The above expression calculates the hash value for the string 'ABC'. To make this hash function predictable, before you run the Python command-line or the IDE, set the environment variable using one of these commands:

```
export PYTHONHASHSEED=0
OR
$Env:PYTHONHASHSEED=0
OR
set PYTHONHASHSEED=0
```

- (A) Draw the contents of hash table (eight slots, 0 to 7), if the collisions are handled by linear probing.
- (B) Assume we want to insert the seventh entry. This entry is yet unknown and the Python's hash function can take every remainder modulo 8 with the same probability. What is the expected number of comparisons before this entry can be inserted into the hash table? (If the entry inserts into an empty slot, it is one comparison; if it needs to do one linear probing to move to the next cell, it is two comparisons; if it does two linear probings, then it is three comparisons, etc.)
- (C) Before the seventh entry was inserted, the hash table was considered full and its size was doubled from 8 to 16 slots. (The new hash function is `hash('ABC') % 16`). Draw the new contents of the hash table containing the same six entries as before.

Answer:

- (A) Here is a short program computing hash function values:

```
>>> S = ["Croatia", "Iceland", "Ireland", "Denmark", "Bulgaria", "Andorra"]
>>> [hash(x) % 8 for x in S]
[7, 4, 2, 0, 1, 6]
```

Note that there are no collisions at all. The hash table looks like this:

T[n]	Entry
T[0]	Denmark
T[1]	Bulgaria
T[2]	Ireland
T[3]	NULL
T[4]	Iceland
T[5]	NULL
T[6]	Andorra
T[7]	Croatia

- (B) If the hash value of the new entry is 3 or 5 (both entries that are currently free) we need just one comparison. If the hash value is 2 or 4 (just one step before a free entry), we need two comparisons. If the hash value is 1, we need three comparisons. If the hash value is 0 we need four comparisons (as the first three slots are filled in). Finally, if the hash value is 7 or 6, we need five or six comparisons respectively (as the linear probing “wraps around” the hash table and starts probing at the beginning).

Here is the expression of the probability:

$$\frac{2}{8} \cdot 1 + \frac{2}{8} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 4 + \frac{1}{8} \cdot 5 + \frac{1}{8} \cdot 6 = \frac{26}{8} = \frac{13}{4}.$$

- (C) Here is the Python snippet we need:

```
>>> [hash(x) % 16 for x in S]
[15, 4, 2, 8, 1, 14]
```

T[n]	Entry
T[0]	Denmark
T[1]	Bulgaria
T[2]	Ireland
T[3]	NULL
T[4]	Iceland
T[5]	NULL
T[6]	NULL
T[7]	NULL
T[8]	NULL
T[9]	NULL
T[10]	NULL
T[11]	NULL
T[12]	NULL
T[13]	NULL
T[14]	Andorra
T[15]	Croatia

Some random items (in our case Andorra and Croatia) have jumped 8 positions ahead in the new hashtable.